面向对象分析与设计
Object Oriented Analysis and Design

—— 结构化软件设计
Structured Software Design

邱明 博士
厦门大学信息学院
mingqiu@xmu.edu.cn
2026 年秋季学期

学习目标与路线
Learning Path

01

结构化设计基础

复杂性、概要设计定位、设计目标

02

DFD → 结构图

变换分析、事务分析、调用层次

03

模块质量

耦合、内聚、深模块、浅模块

主线：问题分解 → 数据流映射 → 模块评价 → 以信息隐藏和接口质量反复校准

PSD: Ch. 1–5

1 基础与流程

从软件复杂性问题到结构化设计流程

SECTION 1



厦门大学信息学院

为什么需要结构化设计

Managing Complexity

软件设计的核心问题不是“把功能写完”，而是让系统长期可理解、可修改。

变更放大

一处小需求引发多处修改
说明依赖没有被隔离

认知负荷

理解一个模块需要掌握太多
无关的实现细节

未知的未知

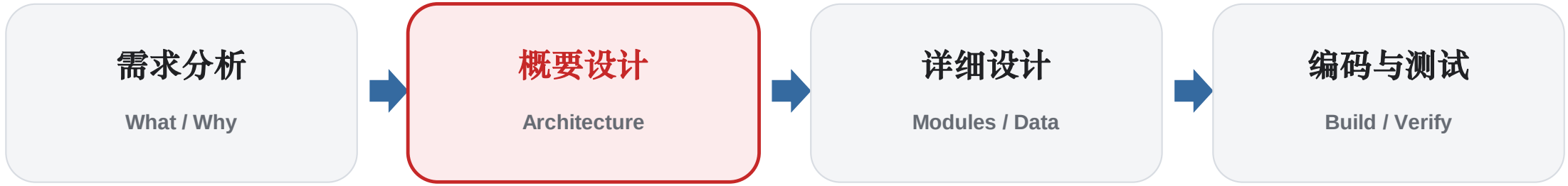
看不出应该修改哪里
系统行为难以预测

目标：降低依赖、隔离变化、缩小理解一个模块所需的上下文。

PSD §2.1–2.3

结构化设计在概要设计中的位置

Architectural Design Context



输入

数据流图 DFD
数据字典
约束与质量目标

输出

软件体系结构
模块结构图与接口
数据/控制关系与设计说明

Structured Design: Yourdon & Constantine

数据流图：结构化设计的输入

Data Flow Diagram Essentials



外部实体

系统边界之外的参与者



处理

把输入变换为输出



数据存储

被读写的持久数据



数据流

在元素之间移动的数据

DFD 检查点

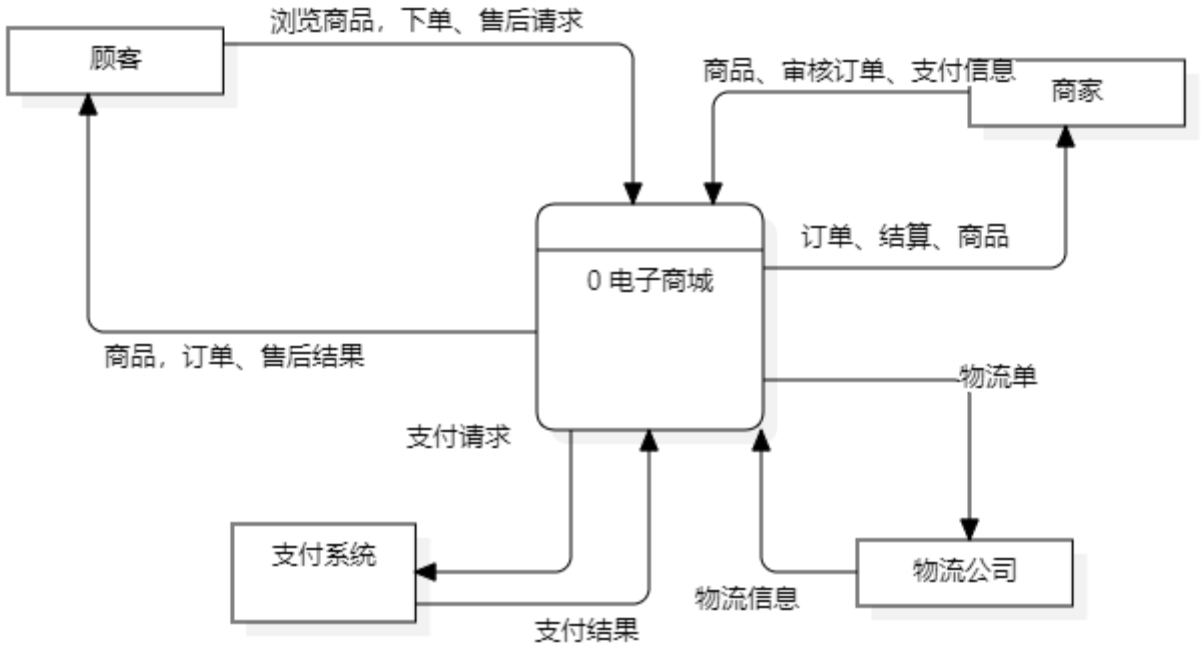
- 每个处理必须有输入和输出，并保持父子图数据流平衡
- 数据流必须有名称和数据类型
- 只描述“做什么”，不提前规定“如何实现”

DFD + Data Dictionary

电子商城 0 级数据流图

Level-0 E-commerce DFD

信息流：顾客、商家、平台管理人员、支付系统、物流系统 ⇿ 电子商城

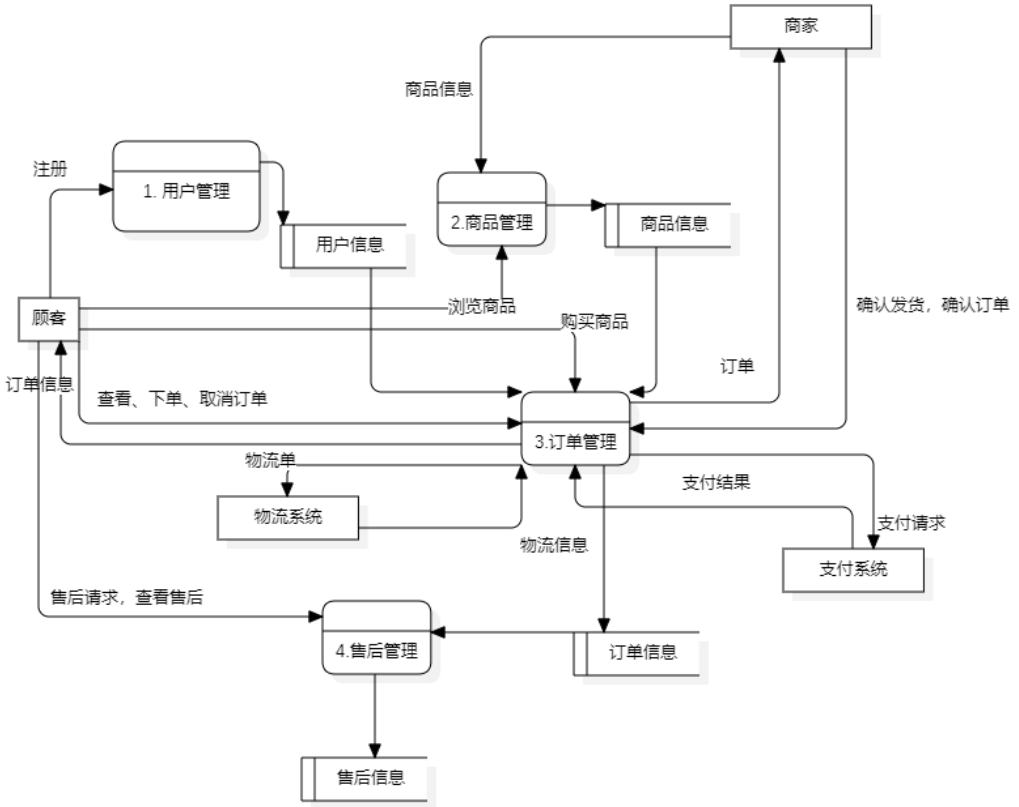


0 级图只展示系统边界，不展开内部处理；所有外部数据流必须与1级图保持平衡。

电子商城 1 级数据流图

Level-1 E-commerce DFD

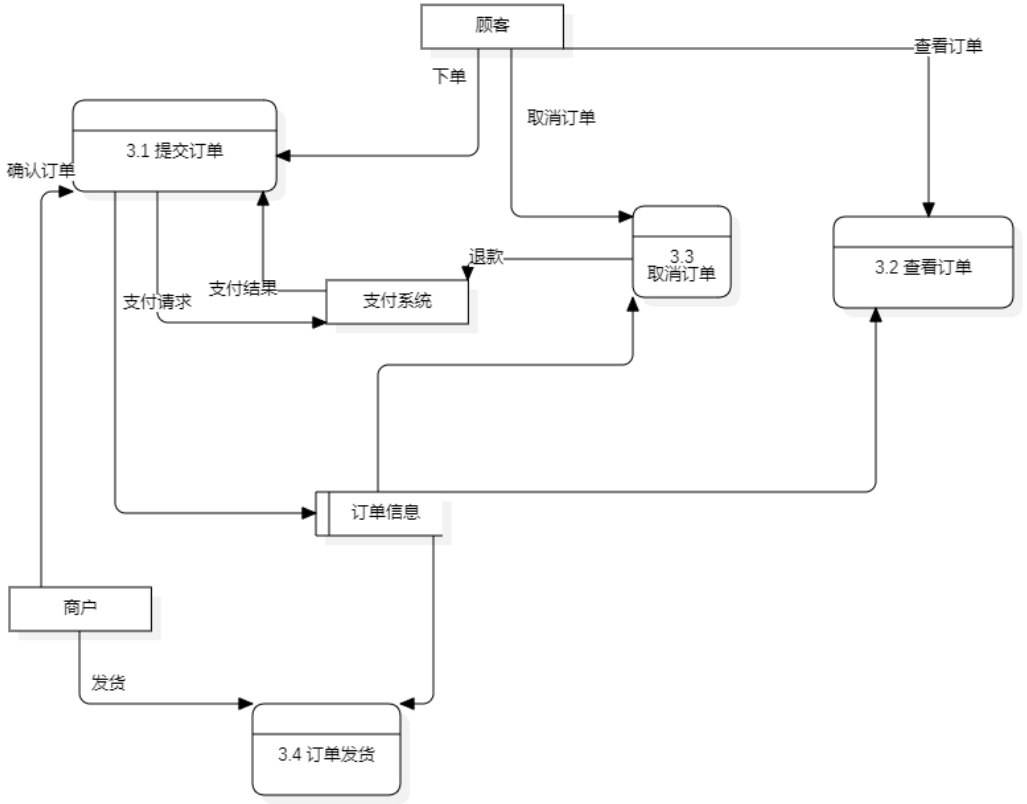
0级图中的“电子商城系统”展开为用户、商品、订单、售后四个部分



1级图由0级图展开；所有输入输出数据流应与第8页0级图保持平衡。

电子商城 2 级数据流图
Level-2 E-commerce DFD

1级图中的“订单管理”展开为提交订单、查看订单、取消订单和订单发货四个部分



2级图由1级图展开；所有输入输出数据流应与1级图保持平衡。

DFD 绘制规则与常见错误

Rules and Common Errors

数据流图描述数据处理，不描述调用顺序、条件分支和程序控制。

处理 Process

必须有输入和输出
名称使用“动词+对象”
例如：校验订单、计算金额

数据流 Data Flow

必须有名称和方向
只写业务数据，不写控制命令
避免无标双向箭头

数据存储 Data Store

表示持久数据
由处理读写
存储之间不能直接连线

父子图平衡 Balance

子图边界数据流必须一致
不能凭空产生或丢失数据
每个数据流应进入数据字典

禁止表达 Control

不画执行顺序
不画决策条件
不画循环和模块调用

常见错误 Errors

黑洞：只有输入没有输出
奇迹：只有输出没有输入
处理过细或一图到底

检查顺序：先核对外部实体 → 再核对数据流平衡 → 最后检查处理命名与数据存储。

2 模块质量

用耦合、内聚和结构指标审查设计

SECTION 3

高内聚与低耦合
High Cohesion, Low Coupling

评价模块设计质量的两个核心标准

高内聚 High Cohesion

- 模块只承担一个明确的业务职责
- 内部方法、数据、规则服务于同一目标
- 模块名称能够概括全部工作
- 修改业务规则时通常只修改本模块
- 内部数据和行为保持一致

目标：功能内聚，避免偶然、逻辑和时间内聚

低耦合 Low Coupling

- 依赖数量少
- 接口范围小
- 只传递完成功能所需的数据
- 依赖抽象，而不是具体实现
- 不直接修改对方内部数据
- 不依赖全局变量和控制标志
- 修改一个模块时，不需要同步修改很多其他模块

模块内部要专注，模块之间要少依赖。

高内聚关注模块内部；低耦合关注模块之间。

耦合：模块之间有多强的依赖
Coupling

内容耦合	一个方法直接访问另一个方法的代码（用goto语句）
公共耦合	多个模块依赖同一个全局数据区（如文件系统，内存数据、外部设备、协议格式等）
控制耦合	方法在调用时了传递控制标志，使得一个方法控制了另一个方法的内部逻辑
标记耦合	传递整个数据结构，但只使用其中一部分
数据耦合	只传递完成功能所需的简单数据
非直接耦合	模块间没有直接数据或调用依赖，通过消息服务器间接调用

设计方向

低耦合

窄接口

只传必要数据

封装易变细节

Low coupling is the goal

耦合：模块之间有多强的依赖
Coupling

公共耦合

多个模块依赖同一个全局数据区（如文件系统，内存数据、外部设备、协议格式等）

```
import java.util.ArrayList;
import java.util.List;

class Mall {
    // 全局共享状态：所有模块都直接访问它
    static final Mall GLOBAL = new Mall();

    List<Product> products = new ArrayList<>();
    List<Order> orders = new ArrayList<>();
    List<String> cart = new ArrayList<>();
    double discount;

    static class Product {
        String sku;
        int stock;
        double price;

        Product(String sku, int stock, double price) {
            this.sku = sku;
            this.stock = stock;
            this.price = price;
        }
    }
     static class Order {
            int id;
            double amount;

            Order(int id, double amount) {
                this.id = id;
                this.amount = amount;
            }
        }
}

class CartModule {
    void addToCart(String sku) {
        // 直接修改全局购物车
        Mall.GLOBAL.cart.add(sku);
    }
}

class AdminModule {
    void setDiscount(double discount) {
        // 直接修改全局折扣
        Mall.GLOBAL.discount = discount;
    }
}
```

耦合：模块之间有多强的依赖
Coupling

```
class OrderModule {
    void submitOrder() {
        Mall mall = Mall.GLOBAL;
        double total = 0;

        for (String sku : mall.cart) {
            Mall.Product product = mall.products.stream()
                    .filter(p -> p.sku.equals(sku))
                    .findFirst()
                    .orElseThrow();

            product.stock--;      // 直接修改商品内部库存
            total += product.price;
        }

        total *= (1 - mall.discount);

        // 直接修改全局订单列表和购物车
        mall.orders.add(new Mall.Order(mall.orders.size() + 1, total));
        mall.cart.clear();
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Mall.GLOBAL.products.add(
                new Mall.Product("SKU-001", 10, 199.0)
        );

        new CartModule().addToCart("SKU-001");
        new AdminModule().setDiscount(0.1);
        new OrderModule().submitOrder();

        System.out.println("订单数: " + Mall.GLOBAL.orders.size());
        System.out.println("剩余库存: " + Mall.GLOBAL.products.get(0).stock);
    }
}
```

耦合：模块之间有多强的依赖
Coupling

控制耦合

方法在调用时了传递控制标志，使得一个方法控制了另一个方法的内部逻辑

```
class Order {
    String id;
    String status;

    Order(String id) {
        this.id = id;
    }
}

class OrderService {

    /*
     * 错误示例:
     * Boolean 参数 isCancel 控制内部执行哪条分支。
     */
    void processOrder(Order order, boolean isCancel) {
        if (isCancel) {
            order.status = "CANCELLED";
            System.out.println("取消订单: " + order.id);
        } else {
            order.status = "CREATED";
            System.out.println("创建订单: " + order.id);
        }
    }
```

```
class OrderController {

    private final OrderService orderService = new OrderService();

    void handleOrder(Order order, boolean isCancel) {
        // 调用者必须知道 true/false 代表什么
        orderService.processOrder(order, isCancel);
    }
}
```

耦合：模块之间有多强的依赖
Coupling

标记耦合

传递整个数据结构，但只使用其中一部分

```
class Customer {
    String id;
    String name;
    String email;
    String phone;
    String address;
    String level;

    Customer(String id, String name, String email,
             String phone, String address, String level) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.level = level;
    }
}
```

```
class EmailService {

    /*
     * 这里只需要 name 和 email，
     * 却接收了整个 Customer 对象。
     */
    void sendWelcomeEmail(Customer customer) {
        String subject = "欢迎，" + customer.name;

        System.out.println("收件人：" + customer.email);
        System.out.println("标题：" + subject);
        System.out.println("发送欢迎邮件");
    }
}

class RegistrationService {

    private final EmailService emailService = new EmailService();

    void register(Customer customer) {
        // 传递整个 Customer，而不是其中真正需要的字段
        emailService.sendWelcomeEmail(customer);
    }
}
```

内聚：一个模块内部有多专注

Cohesion

偶然内聚	模块内各任务没有相关性
逻辑内聚	把逻辑相似的功能合进同一模块，由参数选择
时间内聚	只是需要同一时间执行而被放在一起
过程内聚	按处理步骤组织，但输入输出关联不强
通信内聚	处理相同数据或输出相同结果
顺序内聚	前一步输出是后一步输入
功能内聚	所有成分都为完成一个明确功能服务

设计方向

高内聚

单一职责

围绕数据与决策

名称可概括全部工作

Functional cohesion

内聚：一个模块内部有多专注

Cohesion

偶然内聚

模块内各任务没有相关性

```
public class CoincidentalMethodDemo {

    void executeMiscTasks() {
        // 1. 计算税费：属于财务领域
        double tax = 1000 * 0.13;

        // 2. 手机号脱敏：属于隐私处理领域
        String phone = "13812345678";
        String maskedPhone =
                phone.substring(0, 3)
                        + "****"
                        + phone.substring(7);

        // 3. 生成验证码：属于身份认证领域
        int verificationCode =
                100000 + (int) (Math.random() * 900000);

        // 4. 发送邮件：属于通知领域
        System.out.println(
                "发送邮件到：student@example.com"
        );

        // 5. 写入日志：属于审计领域
        System.out.println("日志：任务执行完成");

        // 6. 洗牌：属于游戏或随机排列领域
        List<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Carol");
        Collections.shuffle(names);

        System.out.println("税费：" + tax);
        System.out.println("脱敏手机号：" + maskedPhone);
        System.out.println("验证码：" + verificationCode);
        System.out.println("打乱后的名单：" + names);
    }
}
```

内聚：一个模块内部有多专注

Cohesion

逻辑内聚

把逻辑相似的功能合进同一模块，由参数选择

```
class InputModule {

    /*
     * 错误示例：逻辑内聚。
     *
     * 键盘、文件、网络三种输入在表面上都属于“读取数据”，
     * 但它们的数据来源、异常处理和内部实现完全不同。
     */
    String readData(String sourceType, String source)
            throws IOException {

        switch (sourceType) {
            case "keyboard":
                return readFromKeyboard();

            case "file":
                return readFromFile(source);

            case "network":
                return readFromNetwork(source);

            default:
                throw new IllegalArgumentException(
                        "不支持的输入类型：" + sourceType
                );
        }
    }
}
```

内聚：一个模块内部有多专注

Cohesion

时间内聚

只是需要同一时间执行而被放在一起

```
class ApplicationStartup {

    /*
     * 错误示例：时间内聚。
     * 这些操作只在系统启动阶段一起执行，
     * 但它们之间没有共同的业务目标。
     */
    void initialize() {
        initLogger();
        initLocalization();
        initMetrics();
        registerHealthChecks();
        initThreadPool();
    }

    private void initLogger() {
        System.out.println("初始化日志系统");
    }

    private void initLocalization() {
        System.out.println("加载国际化资源");
    }

    private void initMetrics() {
        System.out.println("启动指标采集");
    }
    ……
}
```

内聚：一个模块内部有多专注

Cohesion

过程内聚

按处理步骤组织，但输入输出关联不强

```
class DeploymentService {

    /*
     * 这些步骤必须按顺序执行。
     * 如果顺序改变，部署就会失败。
     */
    void deploy(String version) {
        createBackup(version);
        stopApplication();
        copyNewVersion(version);
        startApplication();
        checkHealth(version);
    }

    private void createBackup(String version) {
        System.out.println("备份当前版本: " + version);
    }

    private void stopApplication() {
        System.out.println("停止应用服务");
    }

    private void copyNewVersion(String version) {
        System.out.println("复制新版本: " + version);
    }

    ……
}
```

内聚：一个模块内部有多专注

Cohesion

通信内聚

处理相同数据或输出相同结果

```
class Sale {
    String product;
    String region;
    double amount;

    Sale(String product, String region, double amount) {
        this.product = product;
        this.region = region;
        this.amount = amount;
    }
}


class SalesAnalysis {
    double totalAmount;
    String topProduct;
    Map<String, Double> amountByRegion;

    SalesAnalysis(
            double totalAmount,
            String topProduct,
            Map<String, Double> amountByRegion) {

        this.totalAmount = totalAmount;
        this.topProduct = topProduct;
        this.amountByRegion = amountByRegion;
    }
```

```
class SalesAnalyzer {

    /*
     * 三种统计都使用同一份 SalesData。
     * 它们不是链式依赖，而是围绕同一数据完成不同统计。
     */
    SalesAnalysis analyze(List<Sale> data) {
        double total = calculateTotal(data);
        String topProduct = findTopProduct(data);
        Map<String, Double> amountByRegion =
                groupAmountByRegion(data);

        return new SalesAnalysis(
                total,
                topProduct,
                amountByRegion
        );
    }

    private double calculateTotal(SalesData data) {
        double total = 0.0;

        for (Sale sale : data.sales) {
            total += sale.amount;
        }

        return total;
    }2
     ……
}
```

内聚：一个模块内部有多专注

Cohesion

顺序内聚

前一步输出是后一步输入

```
import java.util.List;

class ScoreProcessor {

    /*
     * 三步形成严格的数据传递关系：
     *
     * readScores() 的输出
     *     -> calculateAverage() 的输入
     *
     * calculateAverage() 的输出
     *     -> printAverage() 的输入
     */
    double processScores() {
        List<Integer> scores = readScores();

        double average = calculateAverage(scores);

        printAverage(average);

        return average;
    }
    ……
}
```

耦合—内聚矩阵：模块边界是否健康?

Coupling × Cohesion

内聚 ↑

低内聚 × 高耦合

职责混杂且依赖紧密
优先拆分与封装

高内聚 × 低耦合

职责清晰且接口稳定
继续用 PSD 检查深度

高内聚 × 高耦合

职责可能合理
但接口或共享知识过多

低内聚 × 低耦合

依赖不高但职责模糊
容易形成浅模块

结构图合理只是起点：还要问“模块是否隐藏了一个稳定而重要的设计决策？”

Structure + information hiding

结构优化：从“能调用”到“好修改”

Optimization Heuristics

提取公共子功能
消除重复调用路径

降低不必要扇出
把协调与执行分开

隔离控制决策
缩小判断的影响范围

隐藏数据格式
避免多个模块共享同一知识

分离通用与专用
避免特殊分支污染核心

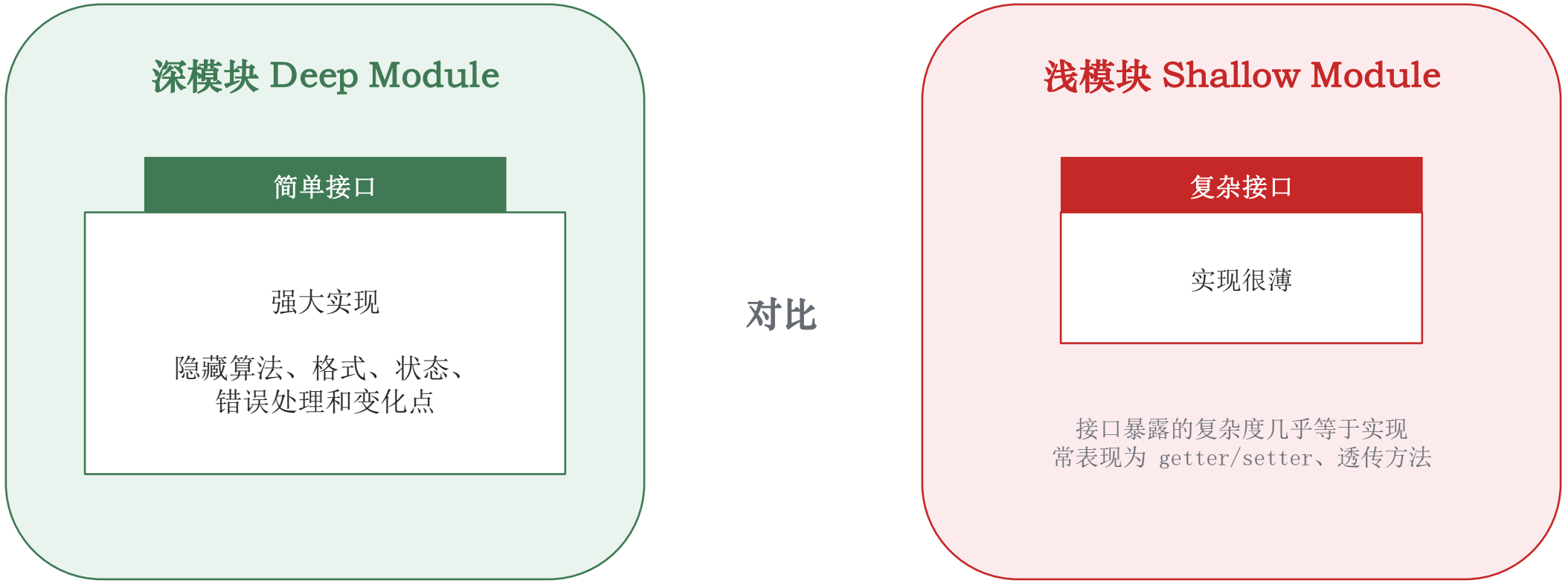
先定义接口与契约
再编写实现细节

消除常见异常路径
让正常流程更直观

检查名称、注释与一致性
让设计意图可读

Refactor the structure, not only the code

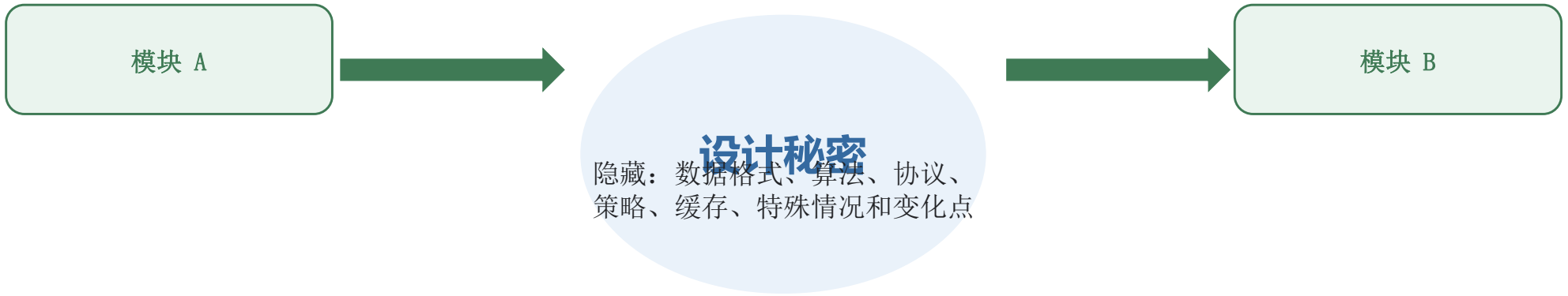
深模块：小接口，大能力
Deep Modules



PSD Ch.4; Red flag: shallow module

信息隐藏：先确定“什么最容易变化”

Information Hiding



时间分解：按执行顺序拆模块，而不是按信息边界隐藏。

信息泄漏：同一个设计决定出现在多个模块，修改时必须同步。

PSD §5.1–5.3; Parnas 1972

不同层次，不同抽象

Different Layer, Different Abstraction

分层抽象

业务策略层 | 决定做什么

领域服务层 | 组织业务规则

基础设施层 | 处理存储与协议

每层提供不同的概念模型，隐藏下层细节。

透传分层

API 方法 | 转发参数

Service 方法 | 继续转发

DAO 方法 | 最终访问数据

接口重复、层次无信息增量，只增加认知负担。

PSD Ch.7; red flag: pass-through method

把复杂性拉向模块内部

Pull Complexity Downwards

最常见用法应该最简单；例外和复杂配置由模块内部处理。

反模式：把选择推给调用者

caller 必须理解缓存、重试、默认值与特殊分支；调用方重复承担复杂性。

推荐：模块吸收复杂性

常见调用零配置；合理默认值和策略封装在模块内部，接口保持稳定。

结构化设计补充：模块接口应窄，但不要把本应内部完成的工作转嫁给上级模块。

PSD Ch.8

通用能力与专用逻辑分离

General-Purpose vs Special-Purpose

适度通用模块

- 围绕核心数据与机制抽象
- 通常拥有更深、更稳定的接口
- 可被多种当前或未来场景复用
- 但不能因为“可能用到”而过度设计

组合

专用适配逻辑

- 处理某一业务规则或界面要求
- 通过组合调用通用模块
- 不把特殊判断渗入核心模块
- 让变化局限在清晰的适配层

PSD Ch.6; §9.4

把错误和特殊情况“定义掉”

Define Errors Out of Existence

异常驱动的设计

删除不存在文件 → 抛异常

获取越界区间 → 抛异常

未提供配置 → 到处判空

结果：正常路径被大量异常分支污染

语义重新定义

删除不存在文件 → 幂等成功

区间越界 → 自动截断到合法范围

缺省配置 → 提供安全默认值

结果：常见情况成为正常语义的一部分

不是吞掉错误；而是减少调用者必须理解的特殊状态。

PSD Ch.10; special cases

设计两次，并用注释检验设计

Design It Twice; Comments First

设计两次 Design It Twice

1. 至少提出两种模块划分或接口方案
2. 比较常见调用是否简单、依赖是否更少
3. 检查哪种方案隐藏了更多设计决策
4. 不一定要写两版代码，但要认真比较设计
5. 在结构设计说明中记录选择理由

先写注释 Comments First

- 接口注释描述契约、约束和常见用法
- 不重复代码，不泄露不必要的实现细节
- 如果接口很难表述，通常说明抽象不清晰
- 名称应当精确、一致、能形成正确的心智图像
- 注释是设计工具，不只是交付文档

PSD Ch.11, 13, 14, 15

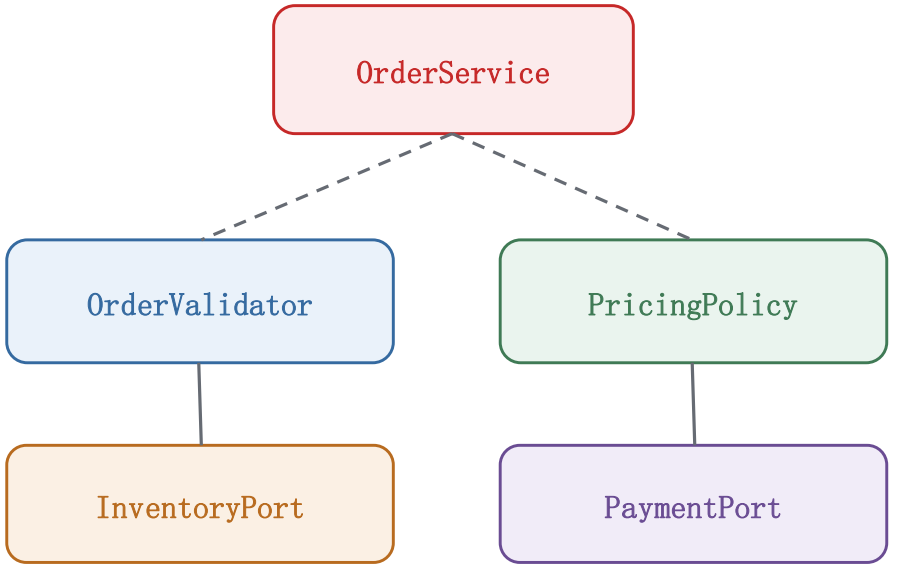
综合案例：从订单 DFD 到模块结构

Case Study: DFD → Modules

DFD 识别



模块结构



接口隐藏规则；结构图控制调用；依赖指向稳定抽象。

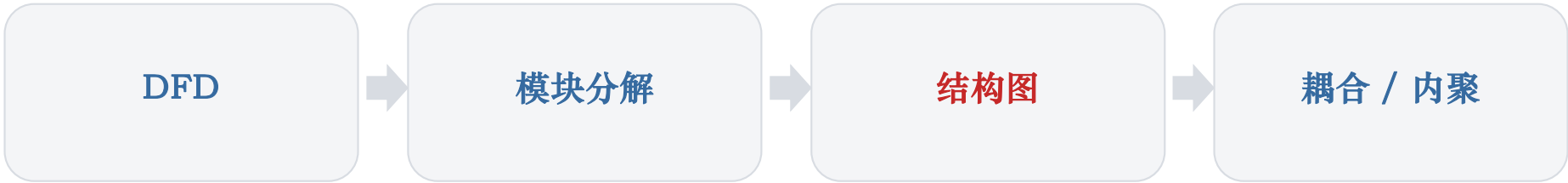
结构化设计质量检查表
Design Review Checklist

- ✓ 模块是否围绕一个明确功能做到高内聚？
- ✓ 模块间是否只传递必要数据，避免公共/控制耦合？
- ✓ 是否隐藏了数据格式、算法、策略等易变决定？
- ✓ 常见调用是否比异常或特殊路径更简单？
- ✓ 通用机制与专用业务规则是否清晰分离？
- ✓ 扇入、扇出、深度与控制范围是否合理？
- ✓ 接口注释能否完整描述契约而不泄露实现？
- ✓ 是否认真比较了至少两种设计方案？

PSD Summary of Red Flags

总结：结构方法 + 设计判断

Summary



结构化设计提供可操作的路径：数据流 → 模块结构 → 质量评价。

PSD 提供模块级判断标准：简单接口、隐藏复杂、消除特殊情况、为阅读而设计。

一句话：好的结构不是“看起来分层”，而是让变化被限制、让常见使用变简单。

References: Yourdon & Constantine (1978); Parnas (1972); Ousterhout, PSD (2018)